Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Лабораторная работа №1

По дисциплине Современные методы искусственного интеллекта

Тема работы Классификация изображений с применением transfer learning

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Сакулин И.М.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Царёв Д.В.
	(дата)	(подпись)	(Ф.И.О.)

2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ **3**

1 БАЗОВЫЕ МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ **4**

1.1 Предварительная обработка данных 4

1.1.1 Бинаризация 4

1.1.2 Исключение среднего 5

1.1.3 Масштабирование 6

1.1.4 Нормализация 6

1.2 Кодирование меток 7

1.3 Логистический классификатор 9

1.4 Наивный байесовский классификатор 10

1.5 Матрица неточностей 12

1.6 Машины опорных векторов 13

1.7 Регрессия 16

1.7.1 Одномерный регрессор 16

1.7.2 Многомерный регрессор 17

1.8 Оценка стоимости недвижимости с помощью SVR 18

ЗАКЛЮЧЕНИЕ **21**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **22**

ВВЕДЕНИЕ

Цель работы — получить практические навыки по обучению распознавания образов, кластеризации и классификации данных с помощью обучения с учителем и метода transfer learning.

Задачи:

- освоить методы предварительной обработки данных средствами `sklearn`;
- изучить кодирование меток;
- построить логистический классификатор и исследовать влияние параметра регуляризации;
- обучить наивный байесовский классификатор и оценить его качество;
- построить матрицу неточностей и проанализировать отчёт о классификации;
- обучить классификатор на опорных векторах на наборе `income_data`;
- построить одномерный и многомерный регрессоры;
- построить SVR-регрессор для оценки стоимости жилья и исследовать влияние гиперпараметров;
- оформить отчёт в соответствии с требованиями.

Все программы, обсуждаемые в работе, доступны в репозитории [1].

1 БАЗОВЫЕ МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ

1.1 Предварительная обработка данных

Входная выборка задана массивом 4×3 (рисунок 1).

```
input_data = np.array([
    [ 5.1, -2.9,  3.3],
    [-1.2,  7.8, -6.1],
    [ 3.9,  0.4,  2.1],
    [ 7.3, -9.9, -4.5]
])
```

Рисунок 1 — Входная выборка 4×3

1.1.1 Бинаризация

Бинаризация выполнялась методом `preprocessing.Binarizer` с порогом 2.1. Результат приведён на рисунке 2.

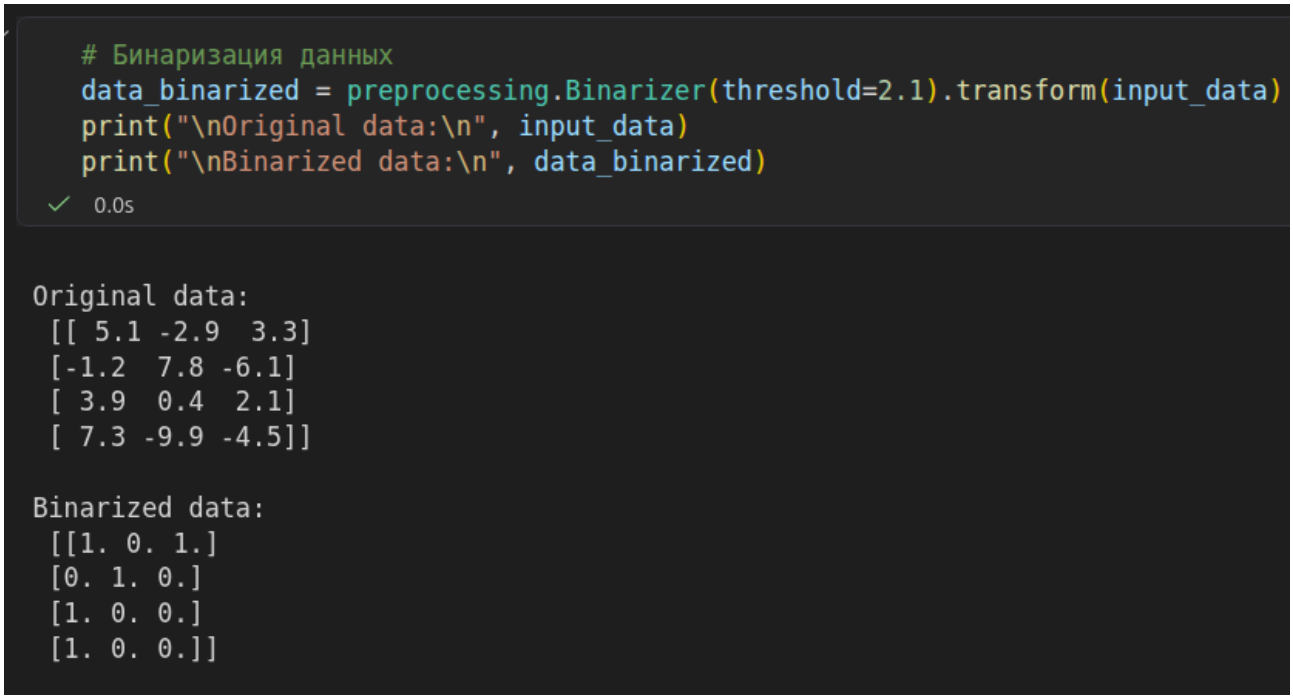


Рисунок 2 — Результат бинаризации

Сопоставляя исходный массив и вывод на рисунке 2, видно, что все значения строго больше 2.1 заменены на 1, а все остальные — на 0. Бинаризация полезна, когда признак имеет смысл «есть/нет», и не подходит, если важна величина отклонения от порога.

1.1.2 Исключение среднего

Среднее и стандартное отклонение вычислялись до и после применения `preprocessing.scale`. Результат приведён на рисунке 3.

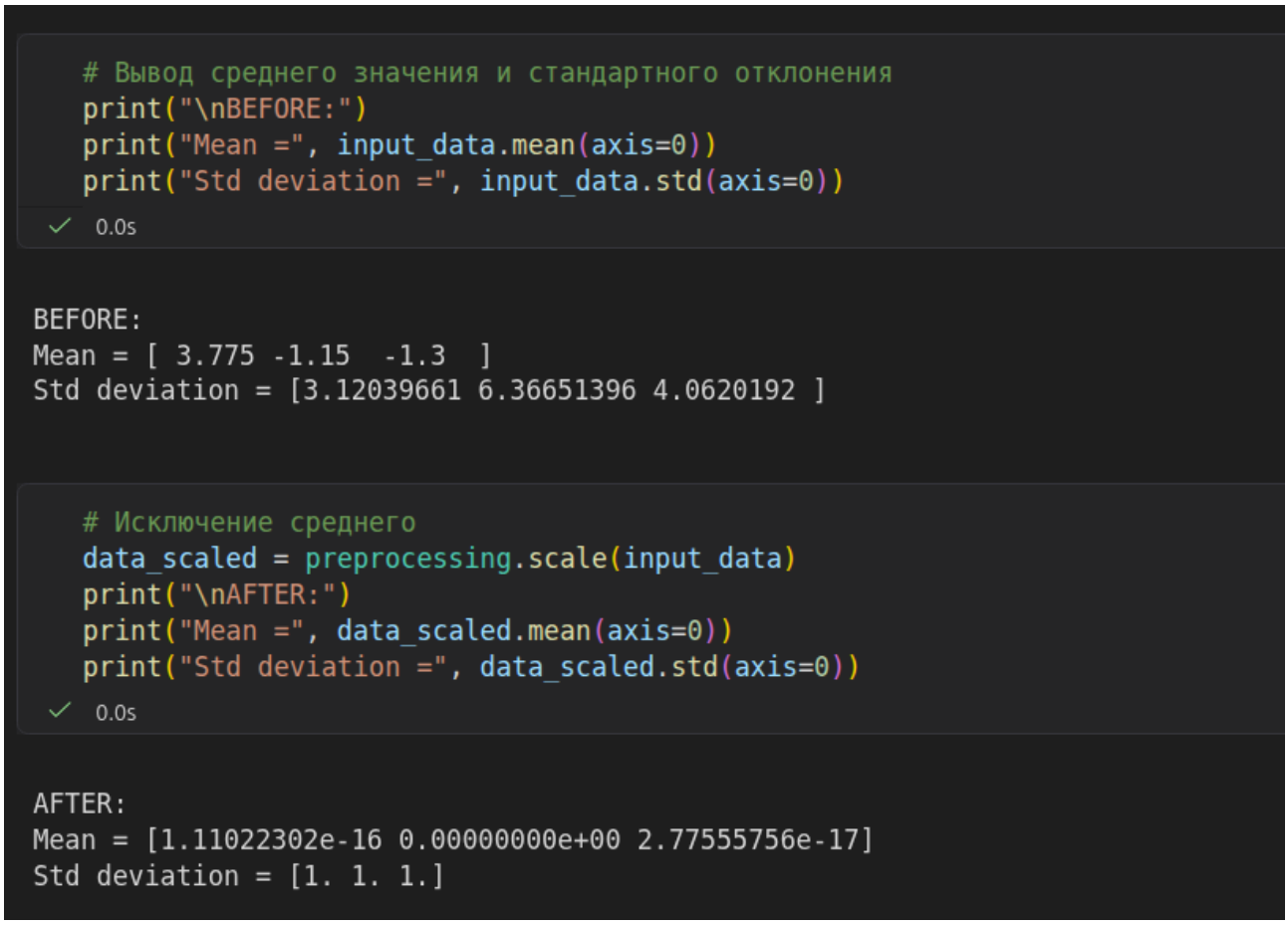


Рисунок 3 — Исключение среднего

До преобразования средние по столбцам сильно отличались от нуля, стандартные отклонения — порядка 3–7. После преобразования, как видно на рисунке 3, средние стали близки к нулю, а стандартные отклонения вовсе стали единицами. Это означает, что каждый признак центрирован и приведён

к одной шкале, поэтому смещение значений больше не влияет на обучение. Обычно эту операцию выполняют перед обучением линейных моделей и SVM.

1.1.3 Масштабирование

Масштабирование выполнялось методом `preprocessing.MinMaxScaler` с диапазоном $(0, 1)$. Результат приведён на рисунке 4.

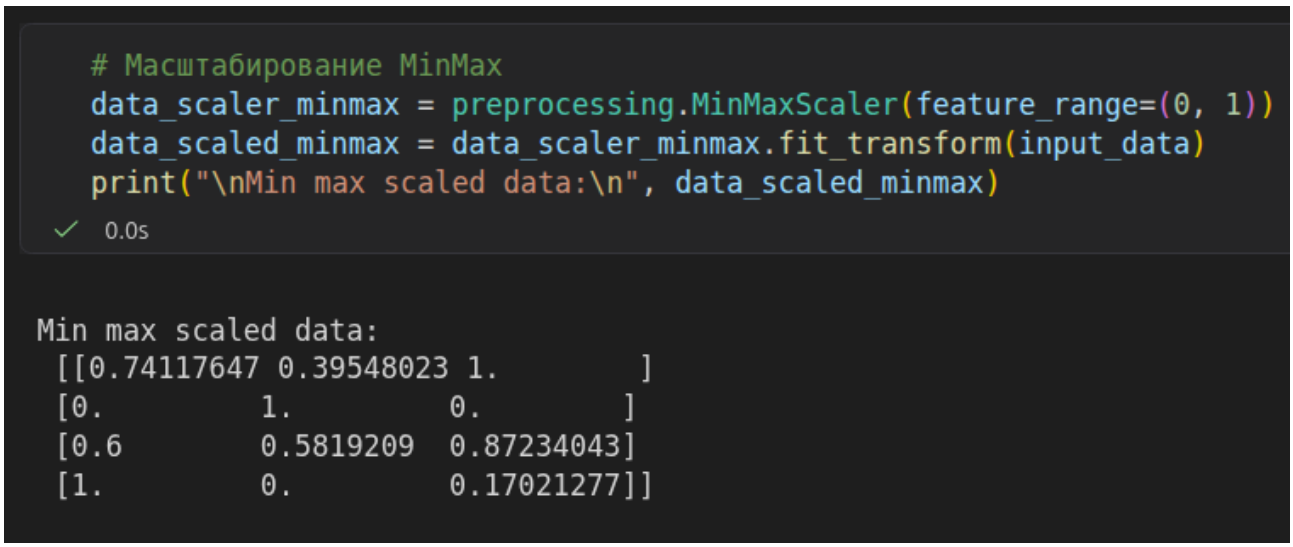


Рисунок 4 — Масштабирование Min–Max

По рисунку 4 видно, что максимальное значение в каждом столбце равно 1, минимальное — 0, а остальные значения линейно отображены в диапазон $[0, 1]$. Такое преобразование полезно, когда признаки измерены в разных единицах, но нужно сохранить форму распределения.

1.1.4 Нормализация

Применены L1 и L2 нормализации методом `preprocessing.normalize`. Результат приведён на рисунке 5.

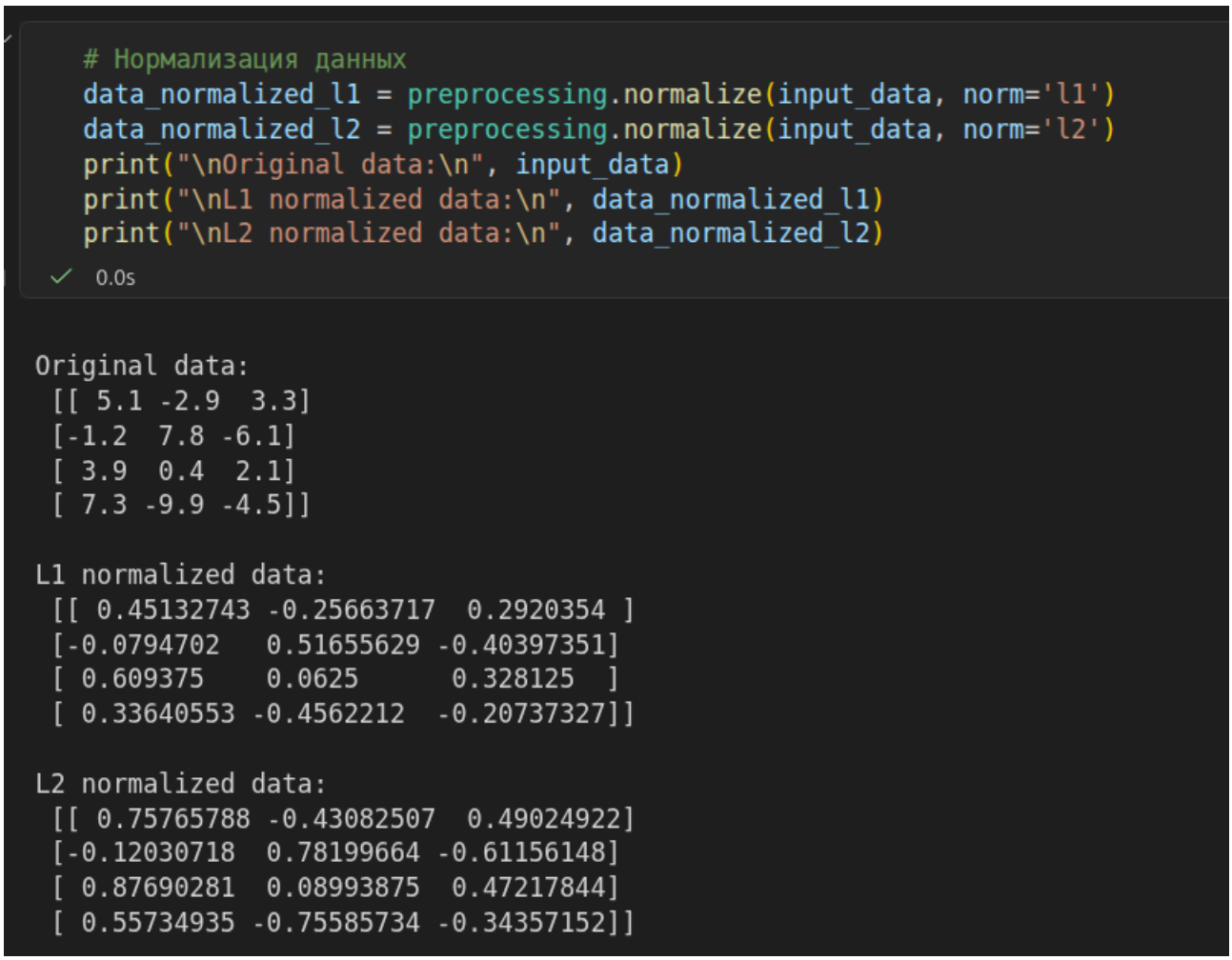


Рисунок 5 — Нормализация L1 и L2

По рисунку 5 можно проверить: для L1 сумма модулей элементов каждой строки равна 1, для L2 сумма квадратов равна 1. Нормализация отличается от масштабирования тем, что она работает со строками (объектами), а не со столбцами (признаками). L1 устойчивее к выбросам, поскольку линейно учитывает вклад каждого значения; L2 сильнее реагирует на большие значения, поэтому её применяют, когда выбросы важны.

1.2 Кодирование меток

Метки преобразовывались в числовой вид через `LabelEncoder`, после чего выполнялось обратное преобразование. Использовался следующий набор меток: `['red', 'black', 'red', 'green', 'black', 'yellow', 'white']`. Результат приведён на рисунке 6.

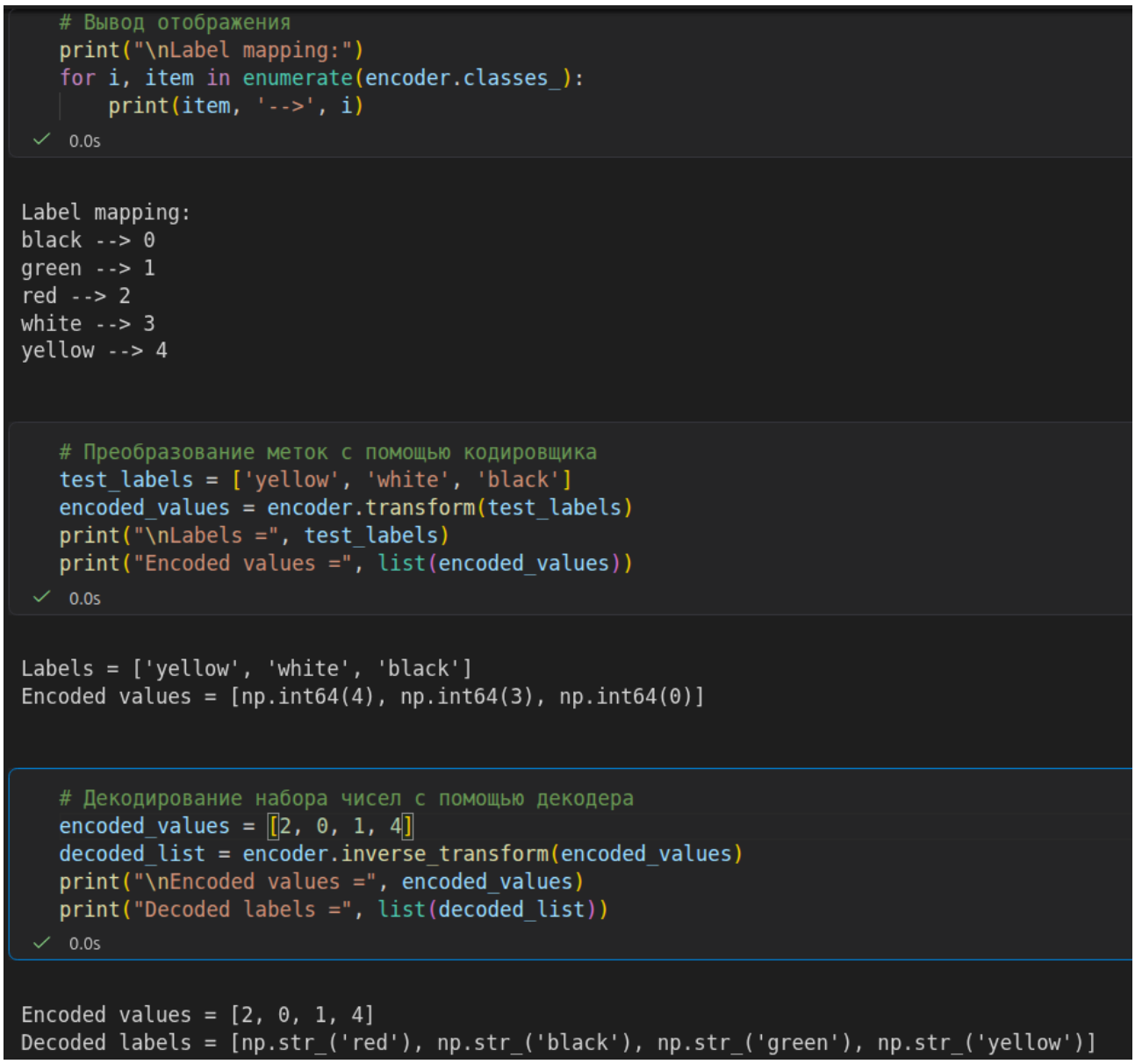


Рисунок 6 — Кодирование и декодирование меток

Как показано на рисунке 6, кодировщик упорядочил уникальные метки по алфавиту и присвоил им индексы. Проверочный набор дал значения, что совпадает с отображением. Обратное преобразование вектора вернуло метки.

Кодирование меток один из способов обработки строковых данных. Еще один способ кодирования меток для несвязанных категорий `one-hot encoding` (с курса машинного обучения) нужен, чтобы модель не воспринимала классы как упорядоченные.

1.3 Логистический классификатор

Логистическая регрессия обучалась на двумерных векторах, соответствующих четырём классам (рисунок 7).

```
X = np.array([
    [3.1, 7.2], [4, 6.7], [2.9, 8], [5.1, 4.5], [6, 5], [5.6, 5],
    [3.3, 0.4], [3.9, 0.9], [2.8, 1], [0.5, 3.4], [1, 4], [0.6, 4.9]
])
y = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3])
```

Рисунок 7 — Входная выборка 4×3

Использовался `LogisticRegression` с решателем `liblinear`. Для анализа изменялся параметр C - параметр обратной регуляризации (обратная штрафу за ошибку величина).

Результат при $C = 1$ приведён на рисунке 8.

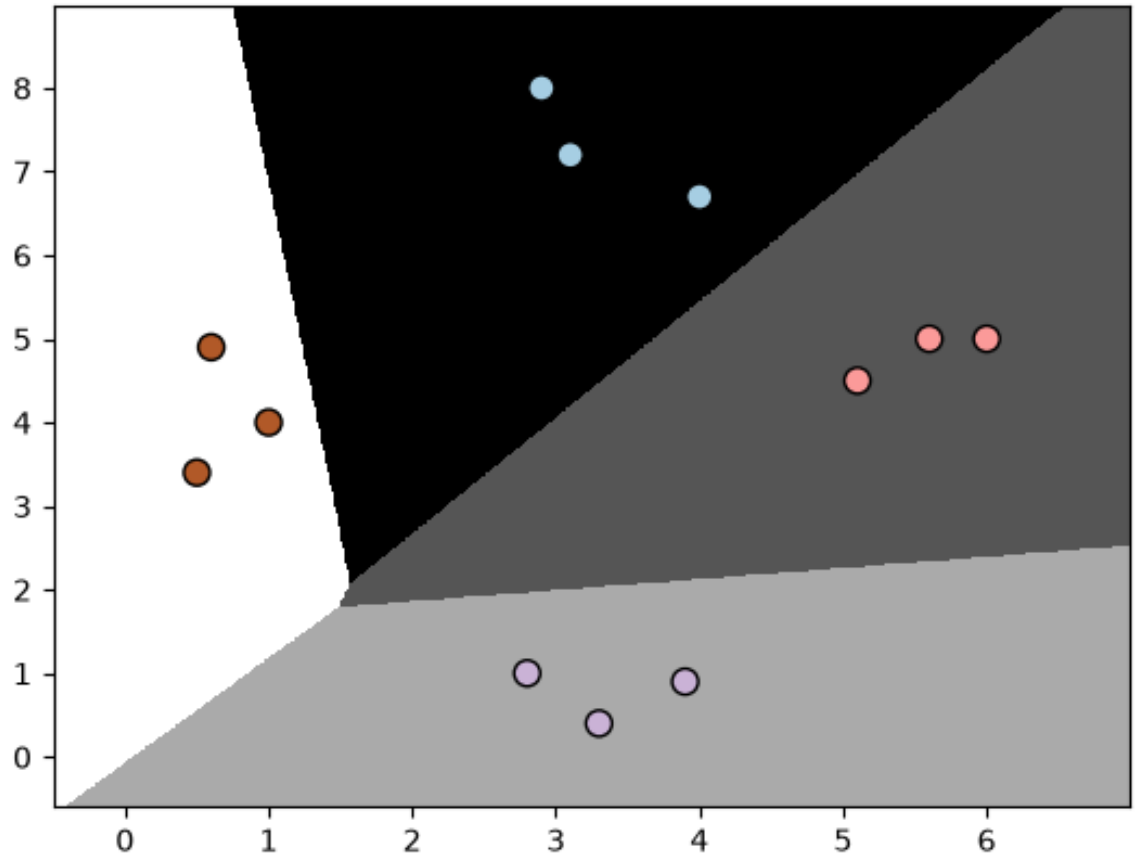


Рисунок 8 — Логистический классификатор при $C = 1$

Результат при $C = 100$ приведён на рисунке 9.

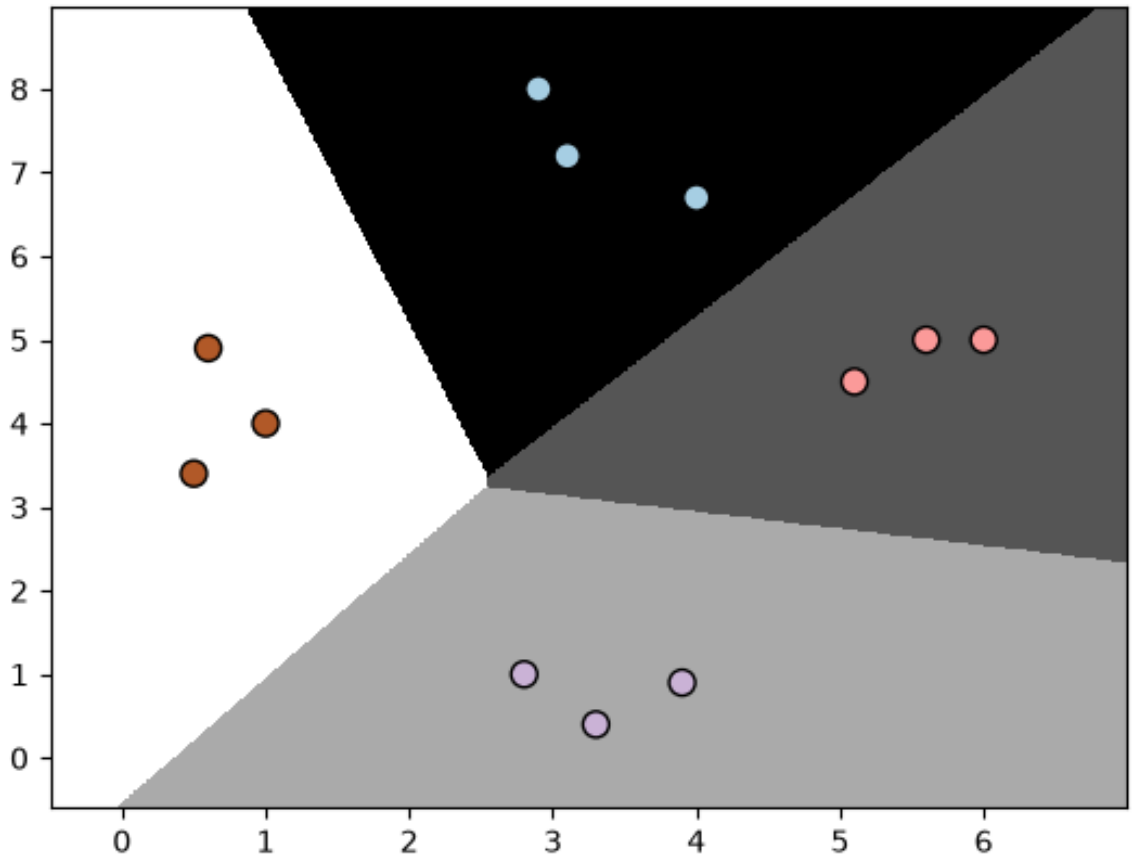


Рисунок 9 — Логистический классификатор при $C = 100$

При увеличении C со 1 до 100 штраф за ошибки классификации возрастает, и модель сильнее подстраивается под обучающие данные. При малом $C = 1$ могло произойти недообучение, на рисунке 8 видно, что границы классов прижимаются к красным и фиолетовым точкам ближе к нулю. При большом $C = 100$ границы стали выглядеть, они равноудалены от точек разных классов. Однако, если задать слишком большой C модель может переобучиться.

1.4 Наивный байесовский классификатор

Использовался `GaussianNB` на данных `data_multivar_nb.txt`.

На рисунке 10 классы разделены корректно, ошибочных точек нет.

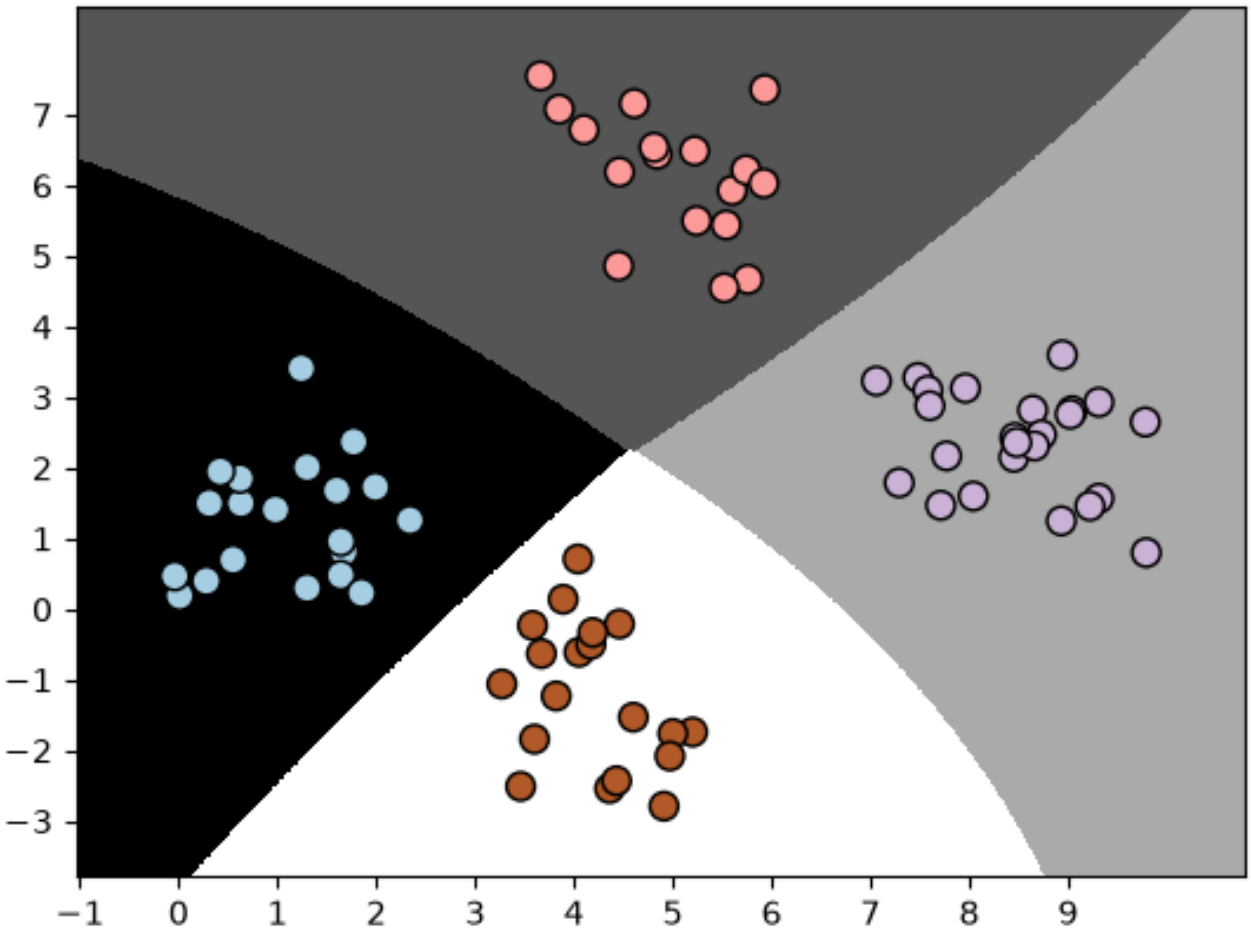


Рисунок 10 — Разбиение данных наивным байесовским классификатором

Метрики на рисунке 11 близки к 100%, precision и recall совпадают — модель сбалансирована и не смещена в сторону одного класса.

```
Оценка показателей классификатора
Accuracy: 99.75%
Precision: 99.76%
Recall: 99.75%
F1: 99.75%
```

Рисунок 11 — Ключевые показатели классификатора

Кросс-валидация даёт более объективную оценку, чем единственное разбиение, поскольку каждый объект хотя бы раз попадает в тестовую выборку.

1.5 Матрица неточностей

Истинные метки: `[2,0,0,2,4,4,1,0,3,3,3]`, предсказанные: `[2,1,0,2,4,3,1,0,1,3,3]`. Результаты приведены на рисунках 12 и 13.

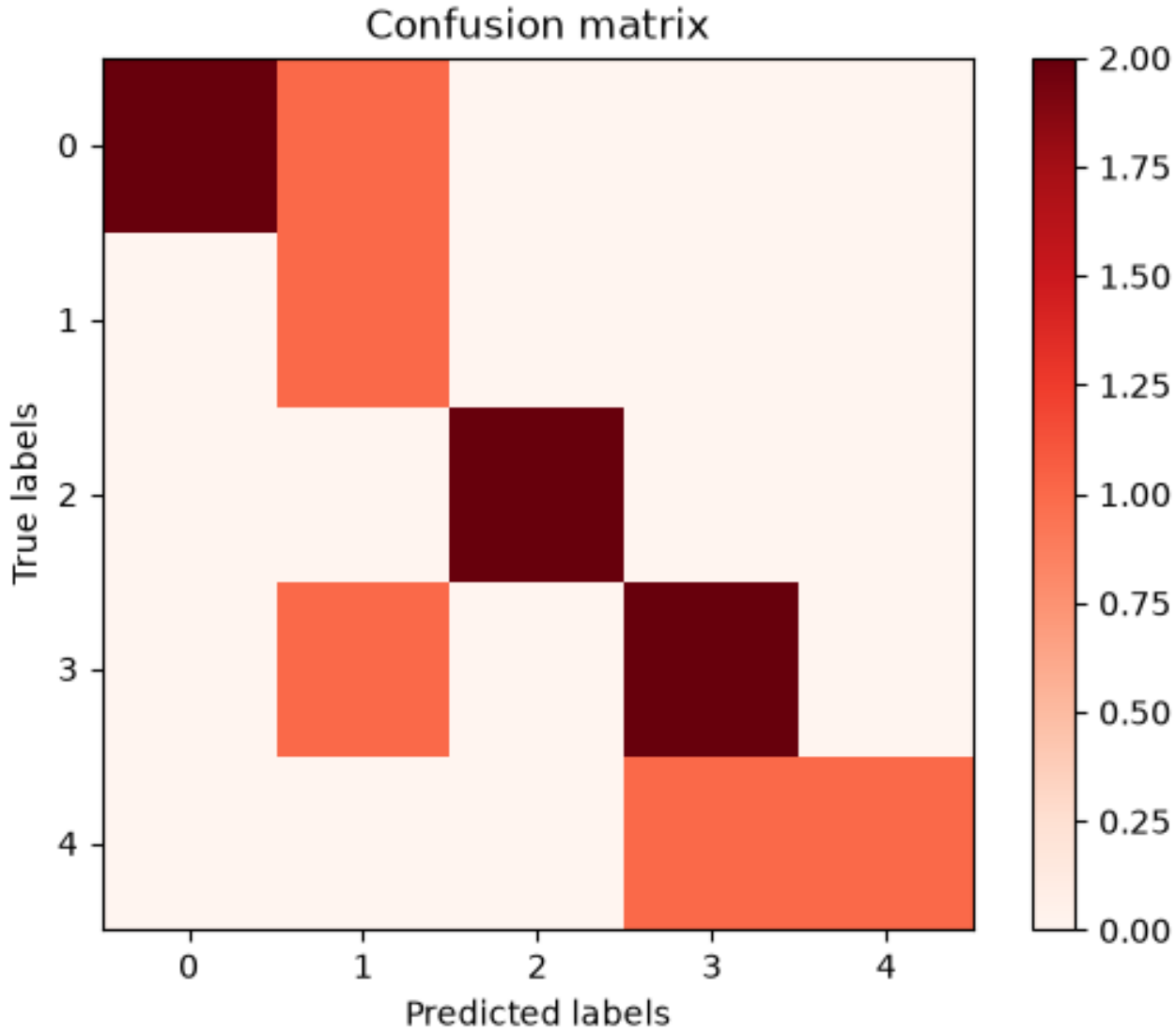


Рисунок 12 — Матрица неточностей

По рисунку 12 матрица имеет размер 5×5, так как мы имеем всего 5 классов. На главной диагонали отображаются корректно предсказанные классы. Все, что больше 0 вне диагонали - ошибки. На текущих данных три ошибки: истинный 0, но предсказано 1, истинный 3, но предсказано 1, истинный 4, но предсказано 3.

```
# Отчет о результатах классификации
targets = ['Class-0', 'Class-1', 'Class-2', 'Class-3', 'Class-4']
print('\n', classification_report(
    true_labels, pred_labels, target_names=targets
))
```

✓ 0.0s

	precision	recall	f1-score	support
Class-0	1.00	0.67	0.80	3
Class-1	0.33	1.00	0.50	1
Class-2	1.00	1.00	1.00	2
Class-3	0.67	0.67	0.67	3
Class-4	1.00	0.50	0.67	2
accuracy			0.73	11
macro avg	0.80	0.77	0.73	11
weighted avg	0.85	0.73	0.75	11

Рисунок 13 — Отчёт о классификации

По рисунку 13 точность (precision) максимальная у классов 0, 2 и 4 (ни одной лишней точки), к классу 1 отнесли две точки не из этого класса, в класс 3 отнесли одну точку не из класса 3. Полнота (recall) максимальная у классов 1 и 2 (все точки найдены), для классов 0 и 3 не было найдено одной истинной точки, для класса 4 не было найдено половины всех точек.

1.6 Машины опорных векторов

Использовался `OneVsOneClassifier(LinearSVC)` на наборе `income_data.txt` для предсказания уровня дохода ($\leq 50K$ или $> 50K$).

На всех данных F1-мера составляет около 76.08%. Это значение можно оправдать тем, что классы несбалансированы (доходов $\leq 50K$ в датасете заметно больше), поэтому линейный SVM не может идеально разделить их в исходном признаковом пространстве.

Далее классификатор проверялся на четырёх точках с разными ядрами и значениями C (рисунок 14).

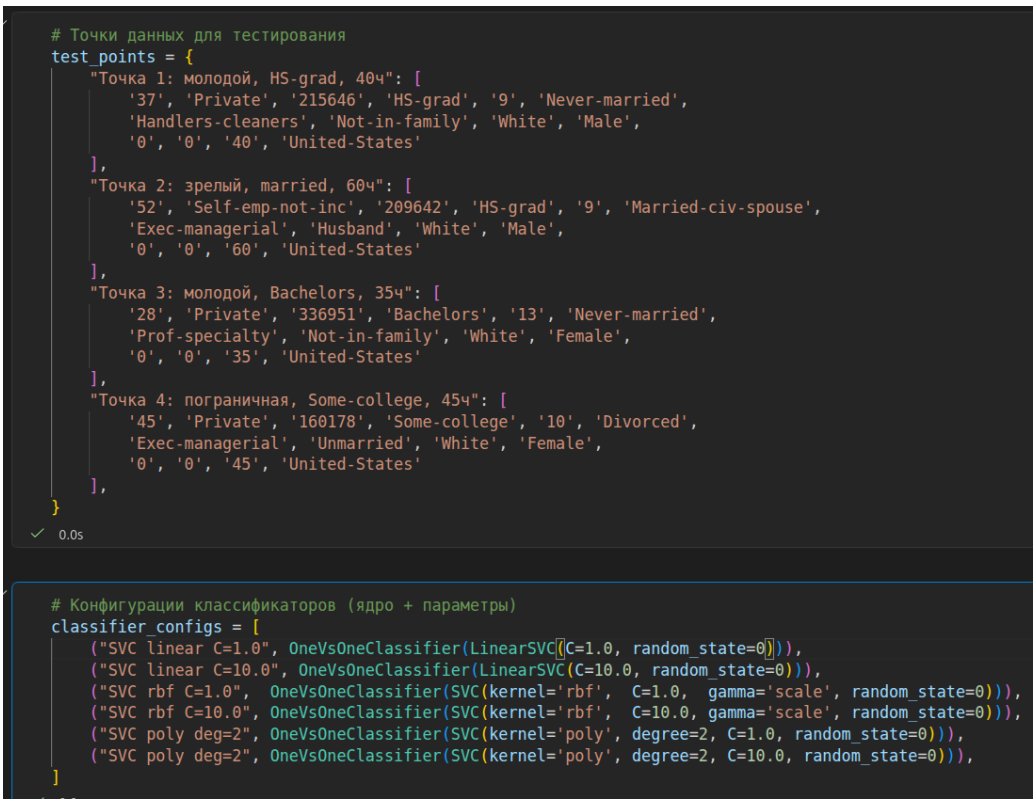


Рисунок 14 — Выбранные точки и ядра

На рисунках 15-17 представлены результаты работы с различными ядрами и точками.

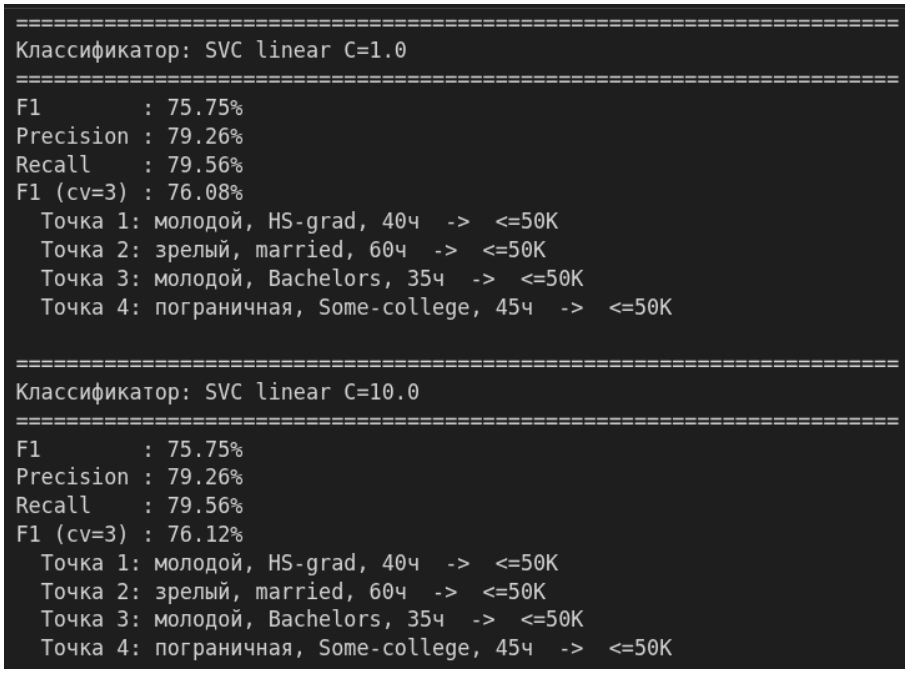


Рисунок 15 — Метрики и прогнозы для линейного ядра

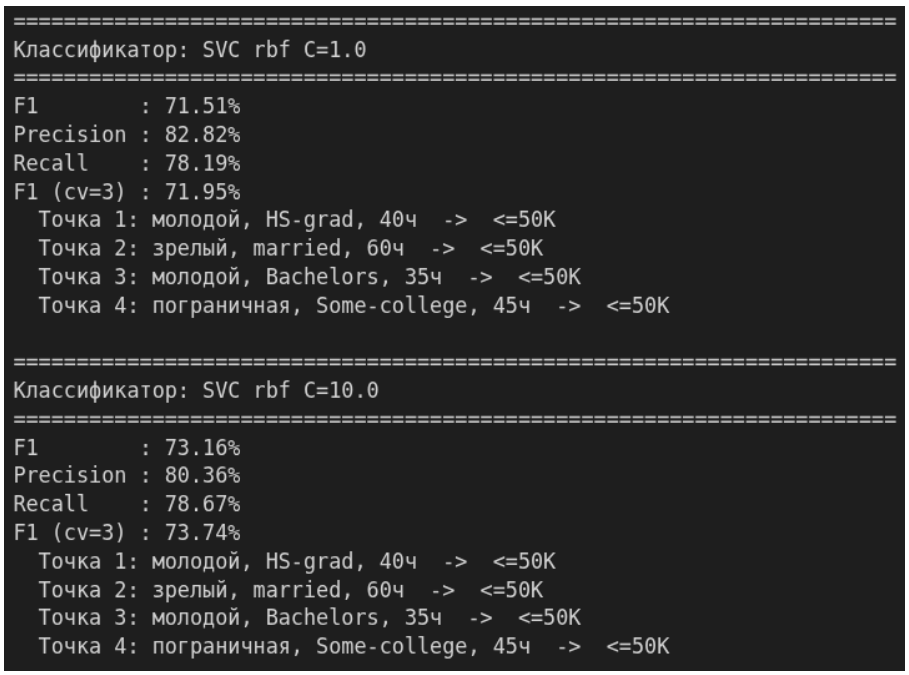


Рисунок 16 — Метрики и прогнозы для радиального базисного ядра

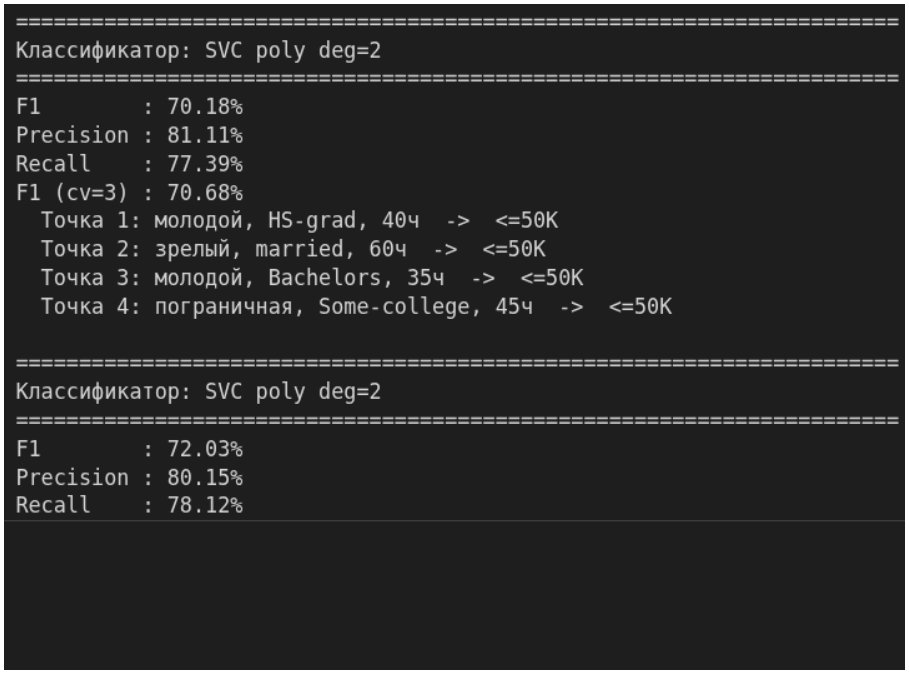


Рисунок 17 — Метрики и прогнозы для полиномиального ядра

Полученные результаты показывают, что линейное ядро дало наилучший результат среди рассмотренных вариантов. При $C = 1$ значение F1-меры составило 75.75%, а при $C = 10$ практически не изменилось и составило 75.75%. При этом значение F1 по кросс-валидации немного увеличилось

с 76.08% до 76.12%, поэтому изменение параметра C в данном диапазоне практически не повлияло на качество модели.

Использование радиального базисного ядра привело к снижению F1-меры до 71.51% при $C = 1$ и до 73.16% при $C = 10$. Полиномиальное ядро также показало более низкий результат: F1-мера составила около $70 - 72\%$. Таким образом, для данного набора данных линейная модель оказалась более подходящей, чем рассмотренные нелинейные ядра.

Для всех проверенных моделей четыре тестовые точки были отнесены к классу $\leq 50K$. При совершенно разных выбранных точках получился одинаковый результат, это объясняется тем, что датасет сильно ассиметричен в сторону $\leq 50K$.

1.7 Регрессия

1.7.1 Одномерный регрессор

На данных `data_singlevar_regr.txt` обучался `LinearRegression`.

На рисунке 19 видно, что облако точек хорошо укладывается на прямую, то есть зависимость близка к линейной. Метрики на рисунке 18 подтверждают это: MAE (среднее отклонение) порядка 3, MSE (средний квадрат отклонения) порядка 15. R^2 около 0.9, близкое к 1, означает, что линейная модель объясняет большую часть дисперсии целевой переменной.

```
Linear regressor performance:
Mean absolute error = 0.59
Mean squared error = 0.49
Median absolute error = 0.51
Explain variance score = 0.86
R2 score = 0.86
```

Рисунок 18 — Метрики одномерного регрессора

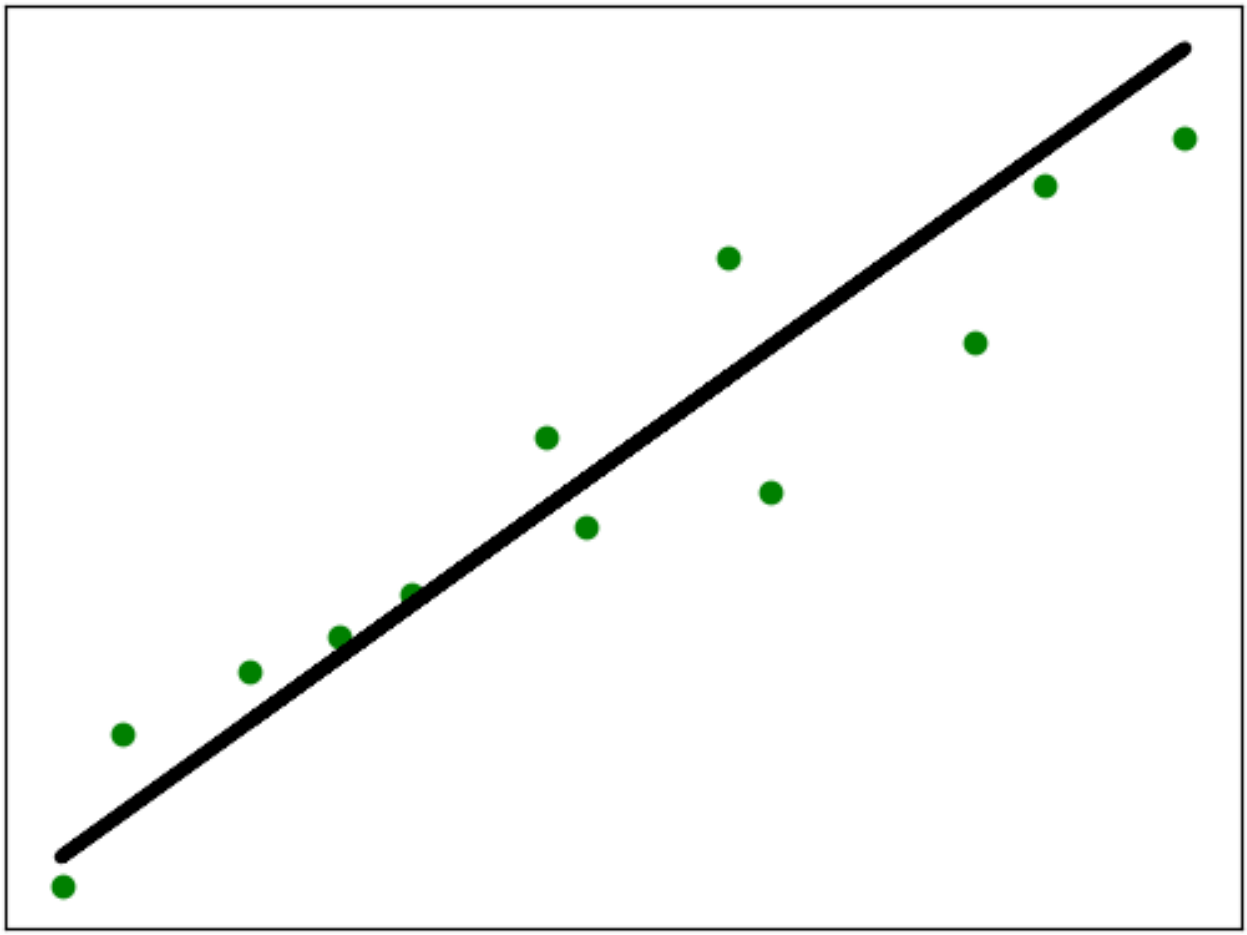


Рисунок 19 — Аппроксимация данных линейным регрессором

1.7.2 Многомерный регрессор

На рисунке 20 представлены результаты оценки линейного регрессора, которые показывают достаточно высокую точность модели и хорошее соответствие предсказанных значений исходным данным.

```
Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86
```

Рисунок 20 — Метрики линейного регрессора

На рисунке 21 представлены результаты оценки полиномиального регрессора, которые свидетельствуют о существенном ухудшении качества прогнозирования по сравнению с линейной моделью.

```
Linear Regressor performance:
Mean absolute error = 11.19
Mean squared error = 907.66
Median absolute error = 5.09
Explained variance score = -4.9
R2 score = -5.04
```

Рисунок 21 — Метрики полиномиального регрессора

На данных `data_multivar_regr.txt` сравнивались линейный регрессор и полиномиальный степени 10. Результат приведён на рисунке **??**.

```
Linear regression:
 [36.05286276]

Polynomial regression:
 [31.63944054]
```

Рисунок 22 — Линейный и полиномиальный многомерные регрессоры

Таким образом, в данном эксперименте линейный регрессор показал более высокое качество на тестовой выборке. Увеличение степени модели до 10 не привело к улучшению результатов: значения ошибок полиномиального регрессора существенно увеличились, а коэффициент R^2 стал отрицательным.

1.8 Оценка стоимости недвижимости с помощью SVR

Для построения регрессионной модели использовался набор данных `California Housing`, содержащий информацию о характеристиках жилых районов Калифорнии. Целевой переменной является медианная стоимость жилья. Были обучены семь моделей `SVR`. Рассматривались линейное и радиально-базисное (`RBF`) ядра с различными значениями параметров C и ϵ. Полученные результаты приведены на рисунке 23.

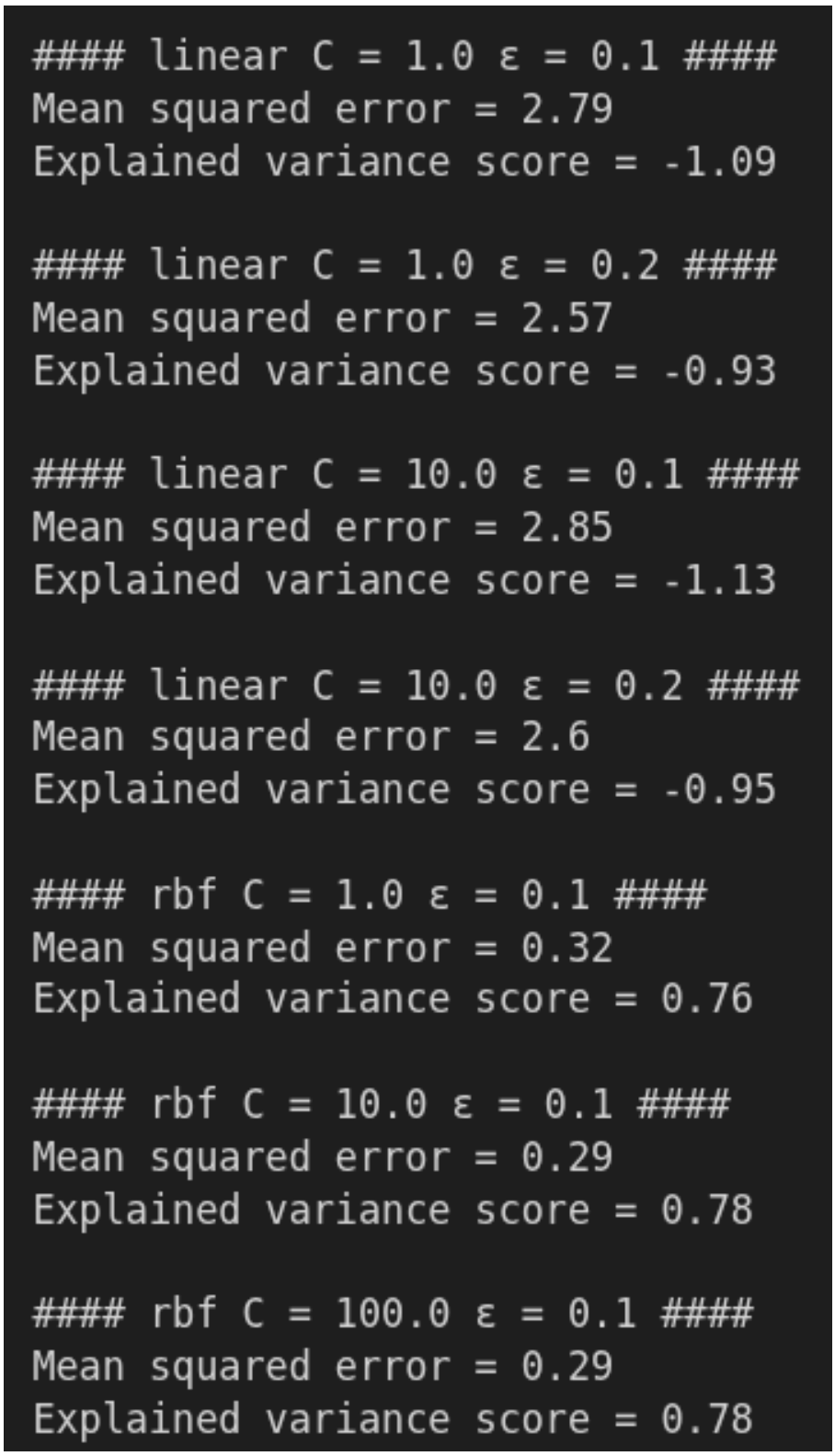


Рисунок 23 — Метрики моделей SVR с различными гиперпараметрами

Модели с линейным ядром показали существенно худшие результаты. Изменение C с 1 до 10 практически не улучшило результаты. Среди рассмотренных моделей наилучшие результаты показали модели с RBF-ядром при

$C = 10$ и $C = 100$. При этом увеличение C с 10 до 100 практически не повлияло на качество модели, поэтому использование $C = 10$ является более рациональным: больше шанс избежать переобучения.

Для дополнительного анализа были выбраны пять тестовых точек, соответствующих различным характеристикам жилых районов. Результаты прогнозирования приведены на рисунке 24.

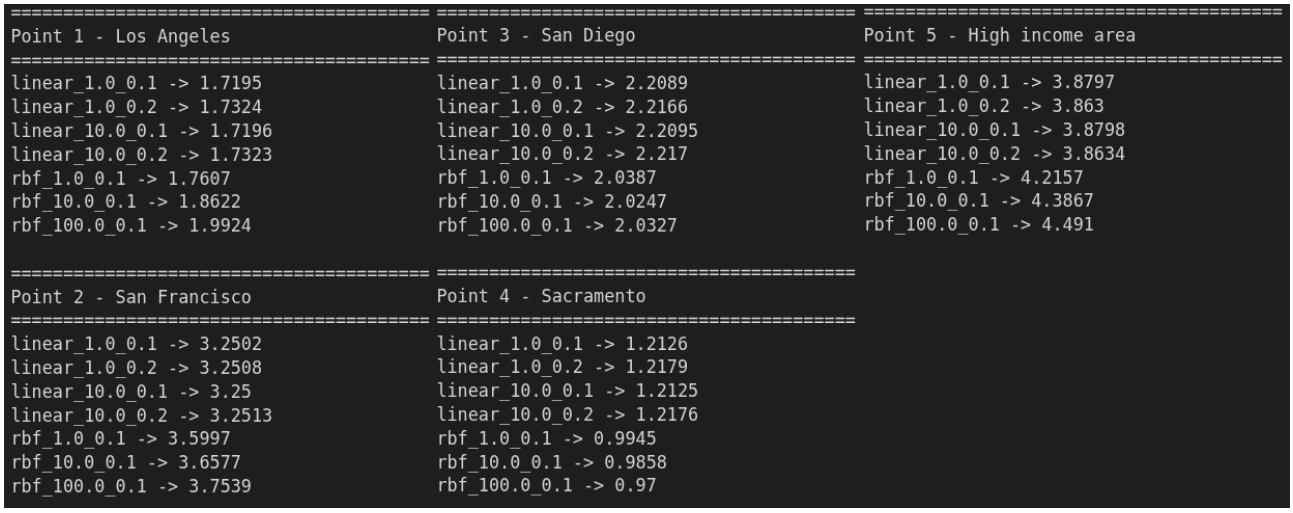


Рисунок 24 — Прогнозы стоимости жилья для тестовых точек

Сравнение результатов показывает, что RBF-модель лучше учитывает нелинейные зависимости между характеристиками района и стоимостью недвижимости. Особенно заметна разница между линейным и RBF-ядром при прогнозировании точек с различными характеристиками. Как и в предыдущем опыте увеличение параметра C в RBF-модели с 1 до 10 заметно улучшило результат, а дальнейшее увеличение до 100 практически не дало прироста качества.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены основные методы предварительной обработки данных, классификации и регрессии с использованием библиотеки `sklearn`. Были рассмотрены логистическая регрессия, наивный байесовский классификатор, SVM и методы регрессии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Исходный код работы [Электронный ресурс]. — URL: `https://github.com/OneTwoZzzPlus/modern-ai-methods/` (дата обращения: 04.10.2026).